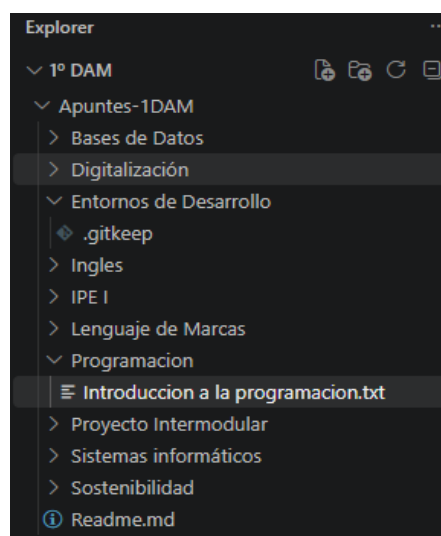


## PRÁCTICA 2 ENTORNOS DE DESARROLLO

Durante el desarrollo de esta práctica, la cual he realizado utilizando lo máximo posible VSCode, he creado un repositorio en GitHub llamado “Apuntes-1DAM”. En dicho repositorio, he organizado en subcarpetas todas las asignaturas del primer curso del ciclo formativo en cuestión. Para ello, me he apoyado en su mayoría de la pestaña de “Source Control” de VSCode dado que tiene varias herramientas donde se integra Git y GitHub, permitiéndome por ejemplo realizar commits, comprobar las diferencias en mi repositorio local con respecto al repositorio de GitHub mediante fetch, etc.

En líneas generales, los pasos que he seguido han sido los siguientes:

- 1º Abrir un directorio nuevo desde la opción “Open folder”. Dicha opción te permite abrir un directorio ya existente en local (es decir, dentro de mi ordenador) o de crear una carpeta nueva dentro de algún directorio ya existente y empezar a trabajar desde ahí. En mi caso, he optado por crear una carpeta nueva dentro de “C:\Users\miguel\1º DAM” llamada “Apuntes-1ºDAM”. Esto me ha ahorrado tiempo dado que es bastante más cómodo que abrir el explorador de archivos de Windows, entrar al directorio antes mencionado y crear la carpeta, para luego abrir el VSCode y abrir el directorio mediante la misma función. De esta manera se hace todo en el mismo paso.
- 2º Crear el árbol de carpetas de las asignaturas dentro del directorio. Aquí, una vez abierto el directorio raíz de “Apuntes-1ºDAM” a la izquierda tenemos la pestaña “Explorer”. Esta pestaña es útil para navegar dentro de los archivos y carpetas del directorio, pero también nos permite crear nuevas carpetas, aunque hay que tener cuidado de seleccionar el directorio raíz correcto para crear una carpeta nueva. Por un lado, objetivamente es mucho más cómodo tener la facilidad de crear una carpeta desde el mismo IDE. Sin embargo, en este caso particular, considero que es mejor hacerlo desde la terminal con el comando “mkdir”. Esto es porque ahí podemos ver de forma clara en que directorio nos encontramos para crear la carpeta nueva, en contraposición a hacerlo desde el propio IDE puesto que debemos tener cuidado de hacer click en el directorio raíz correcto.



- 3º Crear los archivos de texto de cada asignatura. Aquí, desde el mismo IDE, en la pestaña de “Explorer” tenemos la opción de crear archivos nuevos y de escribir que tipo de archivo queremos crear (.txt, .json, .py, etc). De nuevo, navegar por la pestaña de explorer para crear el archivo en el directorio requerido es la forma más rápida, aunque se corre el riesgo de crearlo en el directorio equivocado por un “missclick” y acabar perdiendo más tiempo en borrar el archivo y crearlo de nuevo en el directorio correcto. En mi opinión, la forma más aconsejable, para evitar equivocaciones, es a través de la terminal de comandos.
- 4º Hacer los commits, fetch y push al repositorio de GitHub. Una vez creado todo el árbol de carpetas y archivos, incluido el “Readme”, a la izquierda debajo de la pestaña de “Explorer” tenemos la pestaña de “Source Control”. Esta considero que es mucho más útil dado que te permite ver en tiempo real y de forma clara los distintos commits que se han hecho durante la sesión y tener “muy a mano” opciones para realizar commits y fetch (para comprobar que ha cambiado con respecto al repositorio remoto antes de integrarlo mediante push) y realizar push. Esto nos ahorra bastante tiempo de realizar los comandos en la terminal e interpretar las respuestas que nos da antes de continuar con el control de versiones.
- No obstante, al realizar el push me encontré con el problema que no me subía al repositorio aquellas carpetas que no tuvieran ningún archivo subido. Esto se debió a que GitHub no permite que se suban carpetas vacías a sus repositorios, por lo que, investigando, tuve que emplear la terminal de comandos y crear archivos “.gitignore” para que dichas carpetas se pudieran subir al repositorio. Esto VSCode no tiene ninguna función a simple vista que permita hacerlo o te haga darte cuenta de este fallo al realizar el push.
- La única opción que se podría echar en falta, que hay que ejecutarla desde la terminal, es Git Status, dado que así podríamos ver en tiempo real si vamos adelantados o atrasados con respecto al estado del repositorio (siempre después de realizar un Git Fetch). En líneas generales, la pestaña de “Source Control” nos ahorra mucho tiempo al tener integradas bastantes opciones para control de versiones de GitHub, pero no podemos olvidarnos de la terminal de comandos para realizar algunas funciones esenciales en el control de versiones.

<https://github.com/miguelaroca97/Apuntes-1DAM/tree/main>

